腾讯云Agent Memory节省61% Token提升52%成功率的诀窍：Mermaid无限画布×上下文卸载

原创 腾讯程序员 腾讯技术工程 May 22, 2026, 2:36 AM 广东 225 listened



作者：kentyhuang

本文提出一种基于上下文卸载与结构化图表示的短期记忆压缩方案：通过将完整信息卸载至外部文件系统，同时利用 Mermaid 图语言将任务执行过程转化为可导航的结构化记忆，使完整信息保留于外部、关键状态以高密度形式留存于上下文。在超长Session实验中，该方案最高节省 61% Token，同时把超长Session任务的通过率从 33% 提升到 50%，相对提升52%。从实验结果可以看出，短期记忆压缩能力尤其适合办公提效、创作、研究和编程这类长任务和多任务场景，该能力已随 TencentDB Agent Memory 产品化节奏发布。

⭐ Github仓库地址：https://github.com/Tencent/TencentDB-Agent-Memory。

引言：什么是“一语胜千言”？

我们每天都在做一件很厉害的事：用一句话概括一整段经历。

朋友问你：“昨天约会怎么样？”你不会从几点见面、点了什么菜开始复述，而是直接说一句：“挺灾难的，他全程都在聊前任。”对方立刻就懂了，甚至还能继续追问细节。这句话不仅是总结，更像一个“压缩包”——点开之后，**里面有情绪、有判断、有故事**。

但 Agent 还不会主动这样解析输入。

当它执行一个长任务时，会不断搜索、读文件、调用工具、修改代码。每一步都在产生大量信息——几千字、上万字。如果这些内容全部堆进上下文，结果往往是：Token 被迅速耗尽，模型被细节淹没，甚至逐渐偏离原本的目标。

问题不在于信息太多，而在于信息没有被“压缩成最可用的形式”。

所以我们做了一件事：**让 Agent 也学会“一语胜千言”。**

TencentDB Agent Memory 通过**上下文卸载 + Mermaid 无限画布**，把长任务过程转化为可管理的结构化记忆：完整信息被保留在外部，关键状态以高密度形式留在上下文中。**在超长 Session 实验中，该方案最高节省 61.38% Token，同时把任务通过率最多提升 51.52%。这一能力也将随 TencentDB Agent Memory产品化及开源。**

接下来，我们从最基础的问题开始：压缩，到底是怎么发生的？

一、压缩的艺术

1.1日常生活中的语言压缩

语言的一个基本能力，是把复杂经验压缩成可传播的符号。

从语言学角度看，词语本身就是一种压缩装置。一个词的表面形式很短，但它能够激活大量背景知识、情绪经验和社会共识。比如“家”并不只是一个居住空间，它还可能包含安全感、亲密关系、责任、归属感等复杂含义；“江湖”也不只是地理意义上的江与湖，而常常指向人情、规则、利益、风险和生存智慧。一个词之所以有力量，正是因为它能用很少的语言承载很厚的语义。

日常表达中，这种压缩更加明显。比如一个人说：

“这场饭局像一场大型狼人杀。”

这句话并没有详细描述饭局中每个人的表情、对话、试探和沉默，但听者往往能迅速理解其中的气氛：表面上大家在吃饭聊天，实际上彼此都在观察、隐藏、判断和博弈。“狼人杀”在这里不再只是一个游戏名称，而成为一个高度浓缩的场景符号。它把一整段复杂的人际互动，压缩成一个可以被快速理解的意象。



这些表达之所以成立，是因为**语言压缩并不依赖字面信息的完整保留，而依赖语义结构的有效保留。**人们在交流时真正需要的，往往不是所有细节，而是能够帮助理解和继续判断的关键结构：发生了什么、重点在哪里、关系如何变化、说话者对此如何评价。

因此，语言压缩可以理解为一个从“经验”到“符号”的映射过程：原始经验往往是连续、庞杂、充满噪音的；而语言会将其切分、筛选、抽象，并封装成较短的表达。好的表达不是信息更

少，而是信息密度更高。它删除了不影响理解的枝节，却保留了能够支撑后续推理的核心语义。

这也是“一语胜千言”的真正含义：一句话并不是神奇地包含了全部细节，而是抓住了最能代表整体的结构，并把它折叠进一个短小、可传播、可展开的符号中。

1.2极致压缩：大模型能理解什么样的"符号"

那么，对拥有“世界通识知识”的大模型来说，什么是压缩？我们能把信息压缩到什么程度？

1.2.1压缩的本质：从稀疏到稠密

对大模型而言，"压缩"只有一个定义——**用更少的 token 表达等量（或近似等量）的语义。**本质上就是把稀疏信息提炼成稠密信息，减少上下文中的字数，同时尽量保持语义不变。

一次搜索返回 2000 tokens 的原文，其中真正有信息量的可能只有 80 tokens 的核心结论。剩下的 1920 tokens 是搜索引擎的 UI 碎片、广告、重复内容、格式标记。这些都是"稀疏"的部分。把它们去掉，只留下核心结论，就是最朴素的压缩。

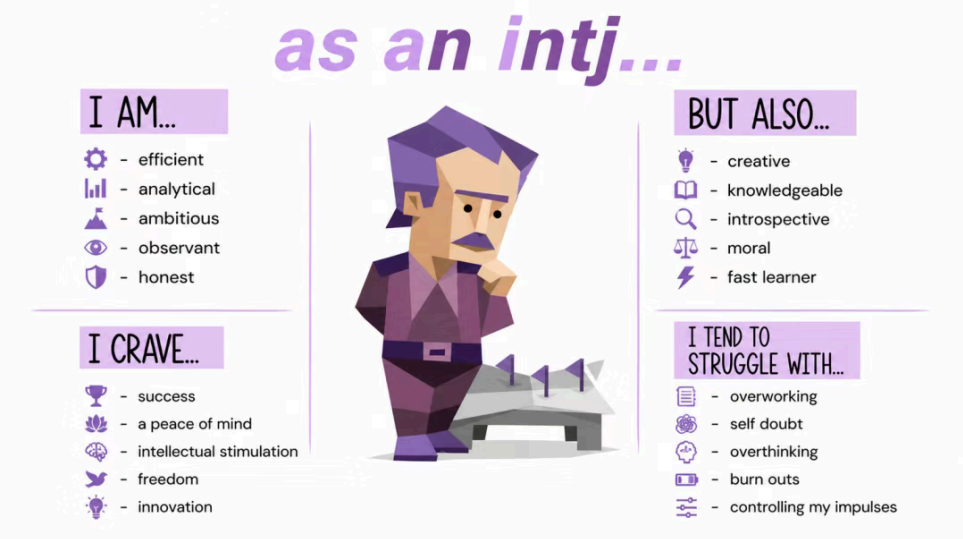
但这还只是文本层面的压缩。**更极致的压缩，是改变信息的表示形式本身。**

1.2.2 大模型对“字 / 画 / 符号”的理解能力

大模型的一个被严重低估的能力是：它不仅能理解自然语言，还能高效理解“符号”。

考虑一个在prompt engineering很火的例子——**MBTI 人格类型**。当你告诉大模型"这个用户是INTJ"，仅仅四个字母，大模型就能立刻理解：

- 内向（I）：偏好独处，深度思考
- 直觉（N）：关注抽象模式和未来可能性
- 思考（T）：决策基于逻辑而非情感
- 判断（J）：偏好计划性和结构化



但这种压缩有一个前提：这个符号已经存在于模型的知识体系中。MBTI 是一种通用知识标签，在训练数据中大量出现，模型早已建立了“INTJ → 一整套语义”的映射关系。本质上，这是一种“记忆驱动”的压缩。

问题在于，这种压缩并不稳定。

一旦换成**模型不熟悉的表达**，比如团队内部的缩写、项目黑话，或者带有语境依赖的符号，理解就会迅速退化。就像人与人之间的“黑话”：在熟人之间可以高效沟通，但一旦脱离语境，就可能完全失效。甚至同一个符号，在不同文化或代际中也可能产生偏差——例如“😊”在不同平台中既可以表示友好，也可能被解读为敷衍或讽刺。

放到 Agent 场景中，这种问题会被放大：如果压缩依赖的是模型未见过的符号，想跨时间、跨agent的沟通就需要额外维护一套“符号对照表”来保证理解一致。这不仅增加系统复杂度，也使跨模型、跨时间的复用变得困难。

因此，并不是所有“短表达”都是有效压缩。

真正可用的压缩，不应该依赖模型“记住了什么符号”，而应该依赖模型“能够从符号中推理出什么结构”。

相比之下，结构化表示（如流程、关系、状态）具有更强的稳定性。它们不是把语义压缩进一个“标签”，而是把信息重组为一种可以被直接解析和推理的形式。因此，大模型往往能比自然语言更高效、稳定地理解结构化表达。



因此，如果我们希望对 Agent 的执行历史进行有效压缩，关键不在于发明更短的“词”，而在于利用一种**既通用（包含在世界知识内）、又可推理的结构化符号系统**。

1.2.3 符号设计的三条原则

经过多次实验，我们总结出三条设计原则：

原则一：符号必须是通用知识。

如果我们发明一套自定义的压缩编码（比如用 T1:HKU:FIN:462K 表示"港大金融硕士 46.2 万"），虽然极其紧凑，但：

- 不同的 LLM 无法理解它（没有在训练数据中见过）
- 同一个 LLM 在不同上下文中可能对它的理解也不一致
- 可适配性和灵活性很差

我们选择的符号载体必须是**所有主流 LLM 都能在预训练阶段大量接触过的通用格式**。

原则二：符号的生成不能过于复杂或孤立。

如果压缩过程涉及复杂的编码规则（比如多层嵌套的缩写体系），生成端和理解端之间很容易产生**语义偏移**——生成符号的 LLM 和后续理解符号的 LLM 对同一个符号的解读不一致。

原则三：表达要足够自由，让模型充分发挥能力。

如果符号系统的规则过于死板（比如"每个节点必须严格包含 5 个字段"），模型在生成时会被格式束缚，无法根据实际情况灵活调整表达。

做到以上三点，基本可以符合模型传递信息用符号压缩的工具属性需求。

二、Mermaid 无限画布：突破物理窗口的认知空间

“符号化压缩”可以把冗长的工具结果压缩成更短、更稠密、更容易被模型理解的符号。

但这只解决了一个问题：“如何压缩单条信息？”

在真实长任务里，Agent 面对的不是一条信息，而是几十次工具调用、多个搜索分支、反复修改的用户要求，以及不断变化的任务状态。

所以更重要的问题是：“当我们有 50 条压缩后的符号时，Agent 如何理解它们之间的关系？”

这就是“无限画布”要解决的问题。

2.1 从“压缩单条信息”到“组织任务结构”

一条 summary 可以很好地压缩一次工具调用。例如，原本80次，几千 tokens 的搜索结果，可以被压缩成一句话：“搜索了港大学费信息，并确认了项目页面中的 tuition 字段。”这对节省 token 很有帮助，但它仍然只是一个孤立符号。

当任务变长以后，我们可能得到很多类似的 summary：

```
{"summary": "搜索了港大学费"}
{"summary": "搜索了港中文学费"}
{"summary": "搜索了港科大学费"}
{"summary": "确认了三所学校的精确学费"}
{"summary": "生成了三校对比表"}
```

这些 summary 虽然比原始工具结果短很多，但它们仍然是线性的。**Agent 可以知道“做过什么”，却不一定能立刻看出“三所学校的搜索其实是并行分支”，“精确学费确认依赖三所学校的数据”，“当前任务正在进入最终汇总阶段”。**

所以，长任务压缩不能只停留在“把内容压短”，还要把压缩后的信息组织成结构。线性列表像日志，适合记录发生过什么；而图结构像地图，更适合告诉 Agent 现在在哪里、从哪里来、接下来往哪里走。

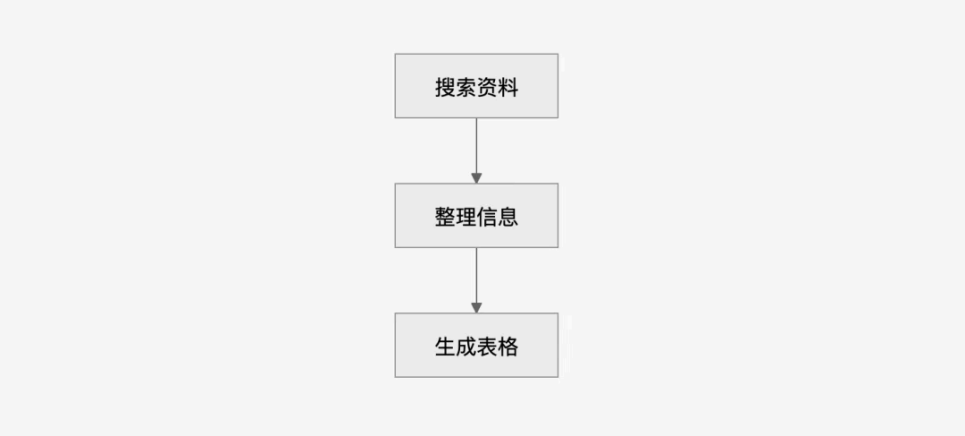
2.2什么是 Mermaid 图：一种模型可读写的符号化图语言

Mermaid 是一种用纯文本描述图结构的标记语言。

比如，我们可以用几行文本描述一个任务流程：

```
flowchart TD
    A["搜索资料"] --> B["整理信息"]
    B --> C["生成表格"]
```

这段文本可以被渲染成图，也可以直接被 LLM 阅读和修改。



所以 Mermaid 有几个很适合 Agent 的特点：

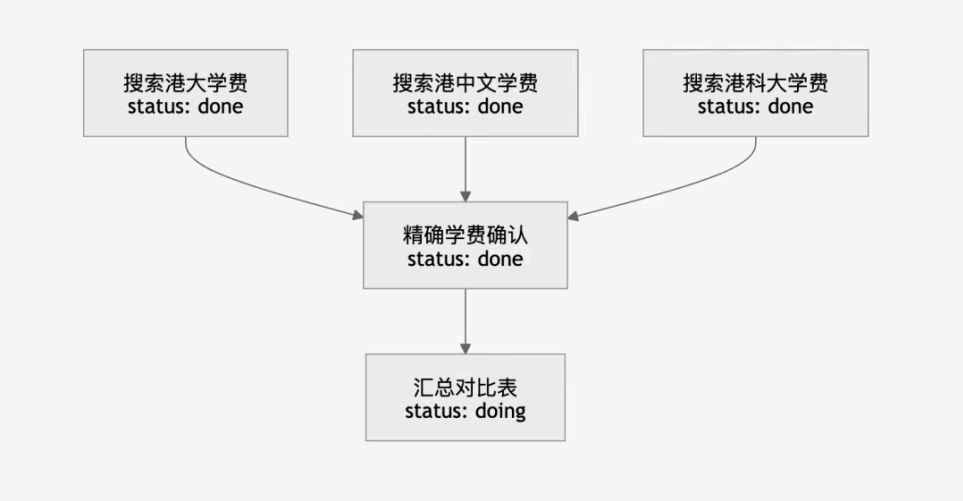
特点	说明
文本化	本质是代码，不是图片，LLM 可以直接读写
结构化	可以表达节点、边、状态、依赖关系
可视化	人类可以渲染成图来检查任务过程
可更新	Agent 可以持续添加、修改、合并节点

特点	说明
可寻址	节点可以绑定 node_id、summary、result_ref
通用性强	Mermaid 是公开格式，不是自创密文

2.3 为什么 Mermaid 适合作为任务画布

Mermaid 的价值不只是“画图”，而是把离散 summary 组织成一张任务拓扑。普通 summary 表达的是“有哪些信息”，而 Mermaid 任务图表达的是“这些信息之间是什么关系”。

例如，三所学校学费整理任务如果只用 summary 表示，Agent 看到的是一串列表。但如果用 Mermaid 表示，结构会清楚很多：



模型可直接阅读的代码模式也相当简洁：

```
flowchart TD
    A["搜索港大学费<br/>status: done"] --> D["精确学费确认<br/>status: done"]
    B["搜索港中文学费<br/>status: done"] --> D
    C["搜索港科大学费<br/>status: done"] --> D
    D --> E["汇总对比表<br/>status: doing"]
```

这张图同时表达了任务结构、当前进度和后续方向。Agent 可以看到，三所学校的搜索都已经完成，它们共同指向“精确学费确认”，而当前正在进行的是“汇总对比表”。这比单纯的线性 summary 多了一层结构信息。

因此，Mermaid 不是把历史变成一段话，而是把历史变成一张可以继续执行的任务地图。它让 Agent 不只是回忆“发生了什么”，还可以判断“下一步应该做什么”。

Mermaid也符合前文提到的符号化设计原则：

原则一：符号必须是通用知识。

Mermaid 语法就是这样一种格式——它在 GitHub、技术文档、教程中广泛存在，所有主流大模型都能准确地读写它。

原则二：符号的生成不能过于复杂或孤立。

Mermaid 的优势在于：它的语法极其简单（节点 + 箭头 + 标签），生成和理解的认知负担几乎一样。一个 LLM 生成的 Mermaid 图，另一个 LLM（甚至同一个 LLM 在不同上下文中）都能准确理解，因为它的语义是由**图的拓扑结构**而非**文本约定**决定的。

原则三：表达要足够自由，让模型充分发挥能力。

Mermaid Flowchart 在这方面近乎完美——它允许任意形状的节点、任意方向的连线、任意长度的标签、任意的嵌套和分支。模型可以自主决定：

- 将连续的 5 个相似操作合并为一个宏观节点
- 为一个关键发现单独建立一个高亮节点
- 用虚线箭头表示弱关联，用实线表示强因果
- 用不同的节点形状区分不同类型的操作

2.4载体选型：Flowchart vs StateDiagram

确定使用 Mermaid 之后，还需要选择具体图类型。我们重点尝试了两种：Flowchart 和 StateDiagram。

两者的核心区别：

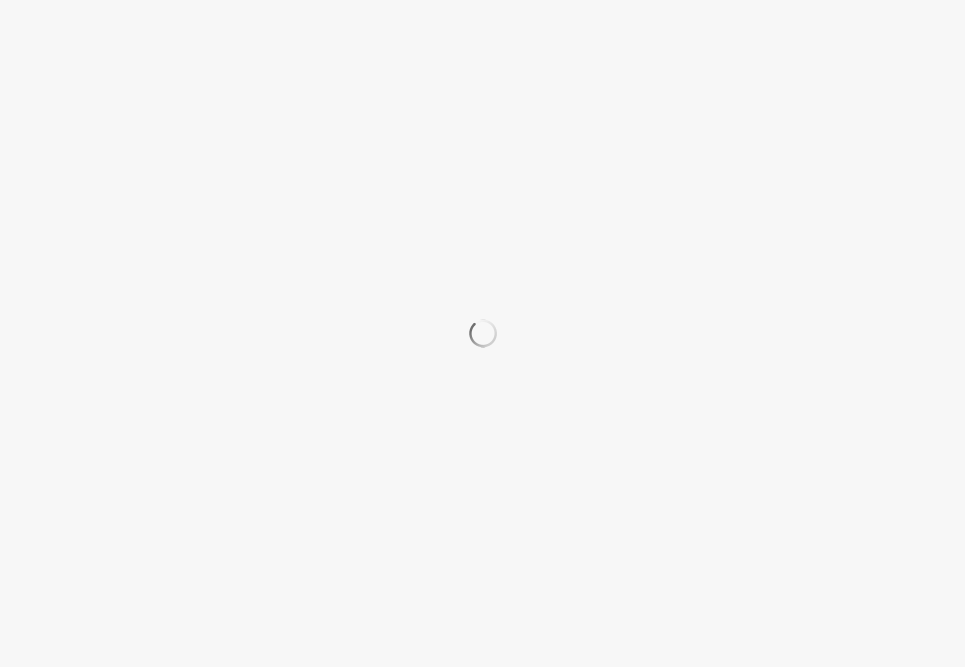
维度	Flowchart（流程图）	StateDiagram（状态图）
核心表达	**过程和结构**	**状态的单向变迁**
拓扑自由度	极高：任意分支、合并、循环、双向箭头	有限：状态之间只有单向转移
节点语义	万能：可以是动作、决策、数据、结论	固定：每个节点就是一个"状态"
类比	**万能流程画布** ：你可以在上面画任何东西	**红绿灯状态机** ：只能从一个状态切换到另一个

先说实验结论：**Flowchart作为压缩载体的效果要比StateDiagram好15%左右。**

StateDiagram 看起来很有吸引力，因为它天然适合表达状态迁移，比如 active、blocked、done 这样的状态。它像一个红绿灯状态机，强调对象从一个状态切换到另一个状态。如果任务本身是严格生命周期驱动的，比如订单状态、审批流、TCP 连接状态，StateDiagram 会非常合适。

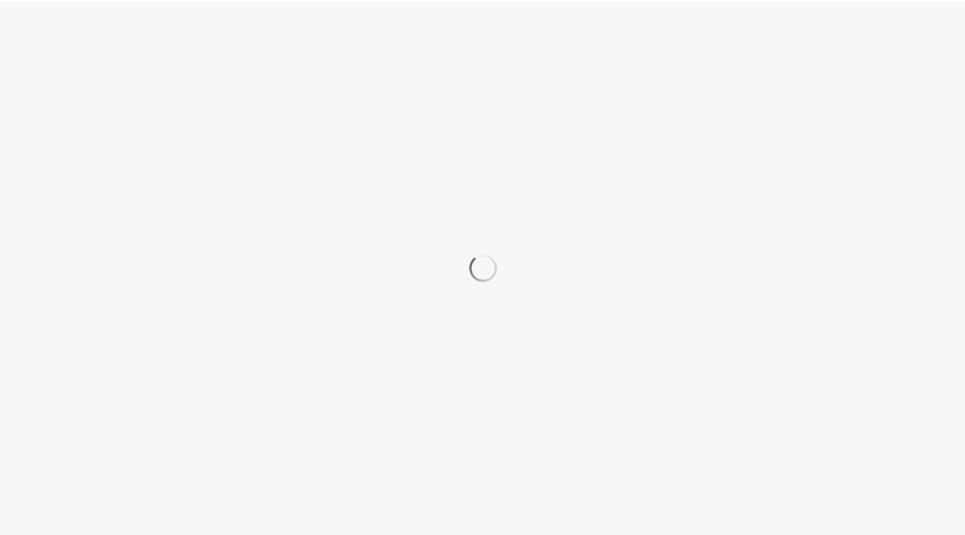
但 Agent 的真实执行过程往往不是这种严格状态机。一个长任务里经常会出现并行搜索、多源汇总、失败回退、交叉引用和临时分支。Agent 并不是沿着一条单线从状态 A 走到状态 B，而是在一个开放空间里不断探索、验证、合并和修正。

比如同样是三所学校学费整理，如果用 StateDiagram 表示，很容易变成线性流程：



这张图能表达顺序，但会弱化一个关键信息：三所学校的搜索其实可以并行，而且“确认学费”依赖的是三所学校的数据汇总，而不是单纯依赖“搜索港科大”这一个前置状态。

如果换成 Flowchart，结构会自然很多：



Flowchart 更像一张万能白板。它可以表达并行分支、汇聚节点、回退路径、参考关系和多步依赖，也允许 LLM 自主决定哪些动作应该合并成一个节点，哪些复杂转折应该拆开表达。

在我们的实验中，**Flowchart 相比 StateDiagram 效果约提升 15%。这说明在长任务上下文卸载场景中，Agent 更需要一张表达复杂依赖和多路径探索且足够自由**的任务地图，而不只是一个状态机。

总的来说：StateDiagram 更适合描述有严格生命周期的对象，**Flowchart 更适合描述 Agent 高度自由的探索式执行过程。Flowchat在“无限画布”的定位中更胜一筹。**

2.5从图到画布：把离散摘要组织成任务拓扑符号化

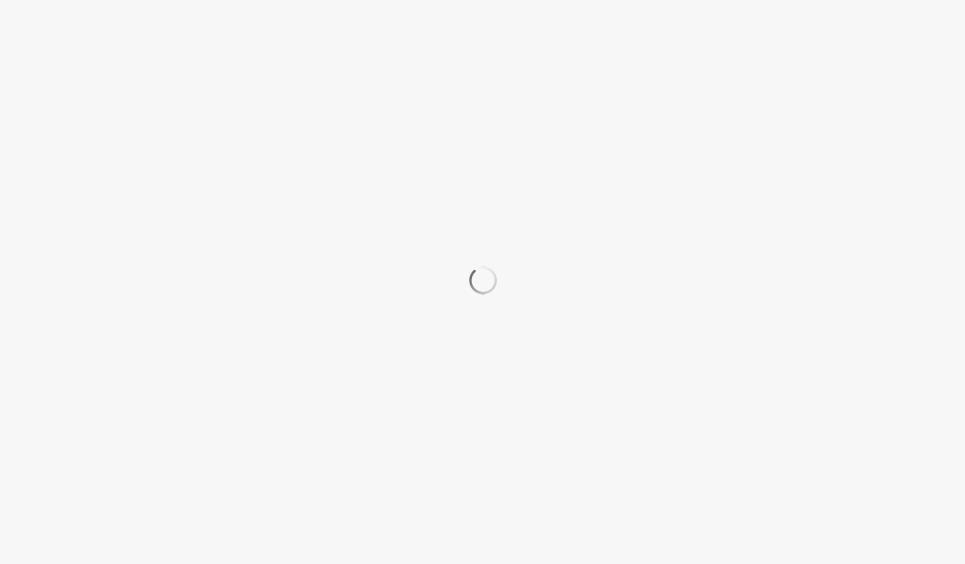
到这里，Mermaid 已经不只是一个图语言，而是变成了一张任务画布。所谓画布，是指它可以持续承载一个任务的结构，并随着任务推进不断生长。

Agent 每完成一组相关动作，就可以在画布上增加或更新节点。例如：

Mermaid 画布可以把这些离散符号组织成拓扑结构：

```
flowchart TD
    N1["搜索港大<br/>status: done<br/>→ 001-N2,N5"]
    N2["搜索港中文<br/>status: done<br/>→ 001-N3"]
    N3["搜索港科大<br/>status: done<br/>→ 001-N4"]
    N4["精确学费确认<br/>status: done<br/>→ 001-N5,N6"]
    N5["汇总对比表<br/>status: doing"]
N1 --> N4N2 --> N4N3 --> N4N4 --> N5
```

解析成图：



这张图用很少的 tokens 表达了三层信息：

信息类型	图中如何体现
任务结构	先搜三所学校，再精确确认，最后汇总
当前进度	前四步 done，正在汇总 doing
符号索引	每个节点关联若干 tool summary，可追溯到 offload.jsonl

这里的关键不是“图更好看”，而是图结构让 Agent 从线性扫描历史，变成结构化理解任务。线性列表适合记录，任务画布适合导航。

2.6不是无限上下文，而是无限可达空间

“无限画布”并不是说模型的上下文窗口真的变成了无限，也不是说 Agent 可以一次性读取无限多的信息。它更准确的含义是：**Agent 的工作空间不再只等于当前上下文窗口，而是扩展成**

了一个可以持续创建、折叠、重新展开的外部认知空间（这里借鉴了上下文卸载和渐进式披露的思想）。

可以把上下文窗口想象成一张办公桌。桌子很方便，因为当前最重要的材料都在眼前；但桌子也有物理大小限制。任务一长，工具结果一多，桌面很快被铺满。传统做法要么把旧材料扔掉，要么把它们压缩成一段很短的总结。这样虽然省了空间，但也带来一个问题：当未来再次需要这些材料时，Agent 可能已经不知道它们在哪里，甚至不知道它们曾经存在过。

Mermaid 画布改变的是这个关系。它不要求所有信息都一直摊开放在上下文里，而是允许 Agent 把任务过程沉淀成一张外部白板：当前任务可以展开，暂时不用的任务可以折叠，未来需要时还可以重新打开。



所以，**“无限”的重点不是“无限读取”，而是“无限可达”**。Agent 不需要把所有历史细节都放进工作记忆里，只需要保留足够清晰的结构入口和路径：知道曾经做过什么、这些任务大概是什么状态、需要时应该从哪里重新展开。

举个很日常的例子：如果你在整理“香港三所大学商学院项目对比表”，中途又被用户要求去查“考研分数线”，再后来又切回“刚才那个港校表格”。如果没有外部画布，Agent 可能需要重新读一遍很长的历史对话，甚至重新搜索资料。但如果每个任务都有一张可以折叠和恢复的画布，Agent 就可以先找到“港校项目对比”这张旧画布，再从原来的任务结构继续往下做。

因此，无限画布的本质可以概括为一句话：

“不是让上下文窗口无限变大，而是让上下文之外的信息仍然可见、可定位、可恢复、可继续执行。”

具体到工程实现中，哪些信息保留在上下文里，哪些信息折叠到外部文件里，以及如何通过索引重新找回，会在后文的“TencentDB Agent Memory 短期记忆压缩方案”章节中展开。

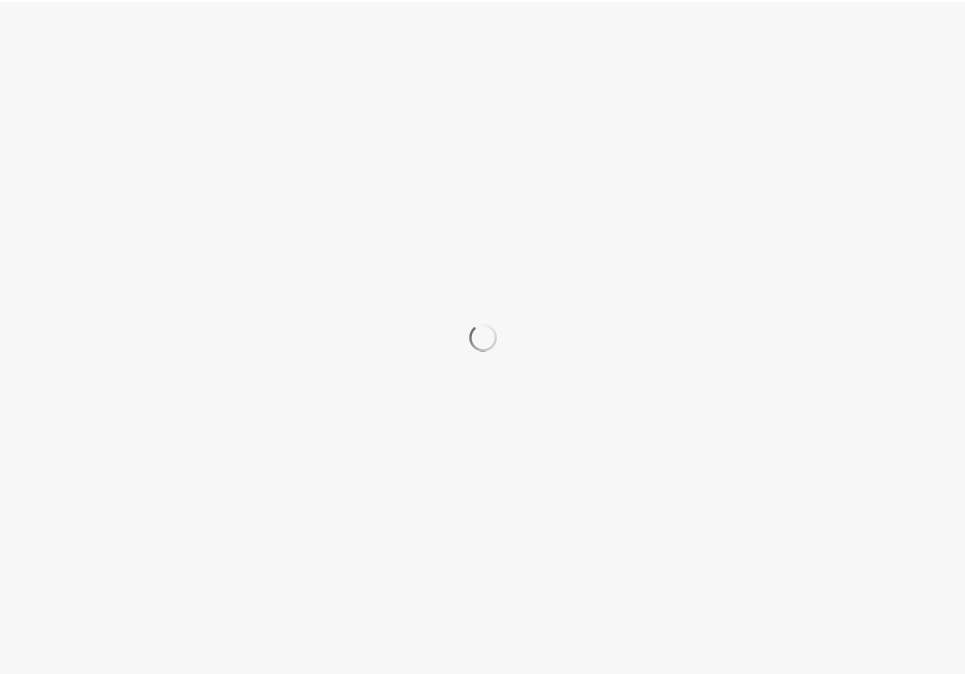
2.7画布如何承载长任务：任务状态的外部化

长任务不是一串简单的聊天记录，而是一个持续变化的任务状态。**Agent 在执行过程中会不断搜索、读取、判断、生成、修改和回退**。如果只把这些过程保存在原始对话里，它们就会变成一条越来越长的线性历史。模型每次都要从头扫描，才能判断任务走到哪里、哪些信息已经确认、哪些路径已经失败。

Mermaid 画布的作用，就是把这条线性历史转化成外部化的任务状态。它不是任务结束后才生成的一张总结图，而是在任务执行过程中持续生长的工作白板。

比如，一个“整理香港高校学费”的任务，刚开始可能只有一个很简单的节点：

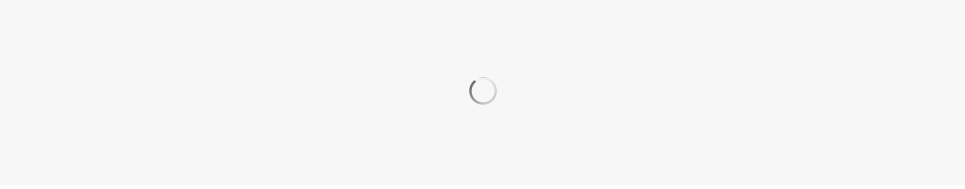
随着 Agent 开始搜索不同学校的信息，画布会逐渐长出来：



这张图表达的不是“每一次工具调用的流水账”，而是“这个任务正在如何展开”。它让 Agent 一眼看出：三所学校的信息已经分别搜索过了，现在正在做口径统一，下一步应该生成对比表。

当任务完成或暂时中断时，画布也不需要一直完整地留在上下文里。它可以从展开状态变成折叠状态，只保留一个较轻的**任务轮廓（类似skill的metadata）**。这个轮廓不负责承载全部细节，而是负责告诉 Agent：这里曾经有一个任务，它大概是什么，当前是什么状态，未来如果需要可以重新打开。

这个过程可以这样理解：



这里最重要的一点是：**折叠不是丢弃**。折叠只是把任务从“正在执行的展开态”转换为“可以恢复的压缩态”。它不再占用大量上下文，但仍然保留了足够的结构，使 Agent 在未来可以重新理解这个任务，而不是从零开始。

从理论上看，画布承担的是一种“任务状态外部化”的角色。它把原本隐含在长对话里的状态，显式地组织成可保存、可折叠、可恢复的结构。这样，Agent 不再只依赖上下文窗口里的线性

历史，而是拥有了一种可以跨轮次、跨任务延续的短期任务记忆。

后文工程实践会进一步解释：我们如何把这种状态外部化具体拆成“原始信息、压缩摘要、任务画布、任务元信息”几层，并通过分级压缩和分级找回来尽量降低信息损失。

2.8 Agent 如何使用画布：层次化注意力机制

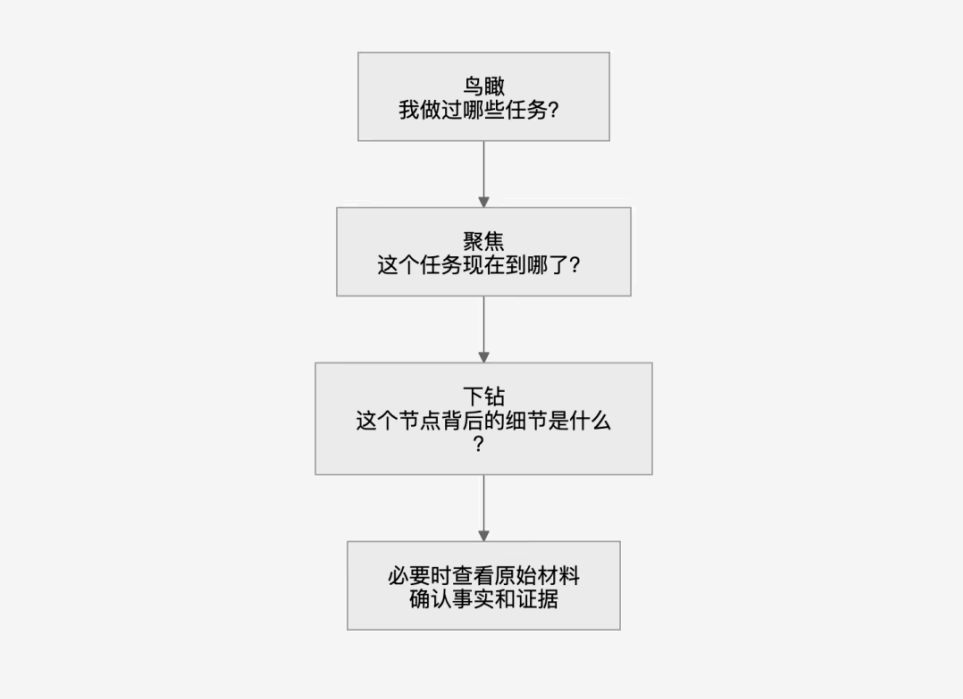
有了无限画布之后，Agent 的信息处理方式也会发生变化。它不再需要把所有历史信息都塞进工作记忆，而是可以像人类处理复杂资料一样，分层查看信息。

第一层是鸟瞰/Overview。Agent 先看到的是任务级别的概览：之前做过哪些任务，哪些已经完成，哪些还在进行，当前用户的问题可能和哪一个历史任务有关。这一层不追求细节，而是帮助 Agent 快速判断方向。

第二层是聚焦。当 Agent 判断某个任务相关时，它再打开对应的任务画布，查看这个任务内部的结构。此时，Agent 看到的不是一长串对话历史，而是一张经过组织的任务地图：哪些步骤已经完成，哪些节点还在进行，哪些信息汇聚成了当前结论。

第三层是下钻/细节补充。当画布上的节点信息仍然不够时，Agent 才进一步追溯到更细的摘要或原始材料。也就是说，原文不是一直常驻上下文，而是在真正需要确认细节时才被重新展开。

可以用下面这张图来理解：

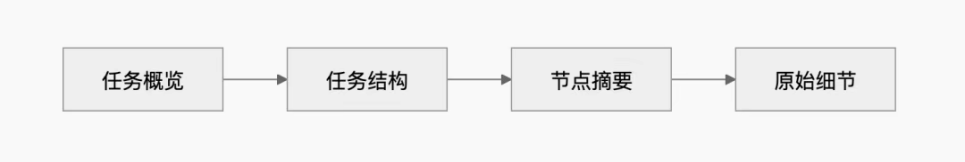


这个过程很像我们自己写报告。我们不会一上来就翻所有原始资料，而是先看目录，知道有哪些章节；再打开相关章节，看结构和重点；最后只有在需要核对细节时，才去翻原始文献或数据表。

对于 Agent 来说也是一样。如果用户说“继续刚才那个港校表格”，Agent 不应该先把所有历史消息读一遍，而是应该先判断这是哪个历史任务，再打开对应的任务画布，找到当前卡住的节

点，最后在需要时追溯具体材料。

这其实是一种层次化注意力机制：



它避免了两种极端：一种是把所有信息都塞进上下文，导致 token 浪费和注意力稀释；另一种是过度摘要，虽然省了 token，却让原始信息无法恢复。

Mermaid 无限画布的价值就在这里。它不是简单地“省 token”，而是改变了 Agent 管理长任务的方式。Agent 不需要记住所有内容，只需要知道哪些信息重要、它们被组织在哪里，以及必要时如何一步步展开。

无限画布的目的不是把上下文变大，而是把 Agent 的注意力变得更有层次。

腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号

三、TencentDB Agent Memory 的短期记忆压缩方案

本章节将硬核拆解在实际落地的AgentMemory产品中的工程逻辑和实践。

在 TencentDB Agent Memory 中，我们把长任务中的短期记忆压缩方案拆分为两部分的结合：

短期记忆压缩 = 上下文卸载 + Mermaid 无限画布。

Mermaid 无限画布，主要解决的是“结构不能丢”的问题：当任务变长、步骤变多时，Agent 需要知道任务走到哪里、哪些步骤有关、下一步应该从哪里继续。

但在真实长任务中，只有任务结构还不够。工具返回、搜索结果、文件内容、日志输出等原始信息往往非常长，如果它们全部留在上下文里，窗口很快会被填满，模型的注意力也会被稀释。因此，我们还需要上下文卸载来解决“信息太长”的问题。

3.1 什么是上下文卸载？

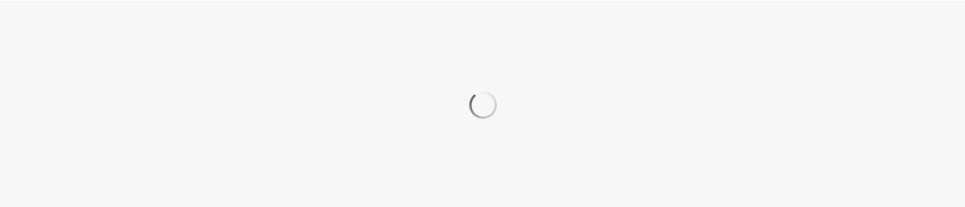
Anthropic的实验显示，当模型的context长度超过max window的80%的时候，会发生“上下文腐烂”，也就是模型的注意力涣散，导致推理能力随之下降。

在长任务里，Agent 最容易被撑爆的不是用户问题，而是“过程信息”：一次网页搜索可能返回几千字，一次文件读取可能返回上万字，一次代码调试可能产生大量日志、diff 和报错。如果

这些内容全部留在上下文窗口里，Agent 就像一张桌子上堆满了资料：东西都还在，但真正要用的那一页反而越来越难找到。

上下文卸载（context offloading）的核心思想很简单：**不是所有信息都必须一直放在模型眼前，给模型赋予信息注意力选择的自主权。**

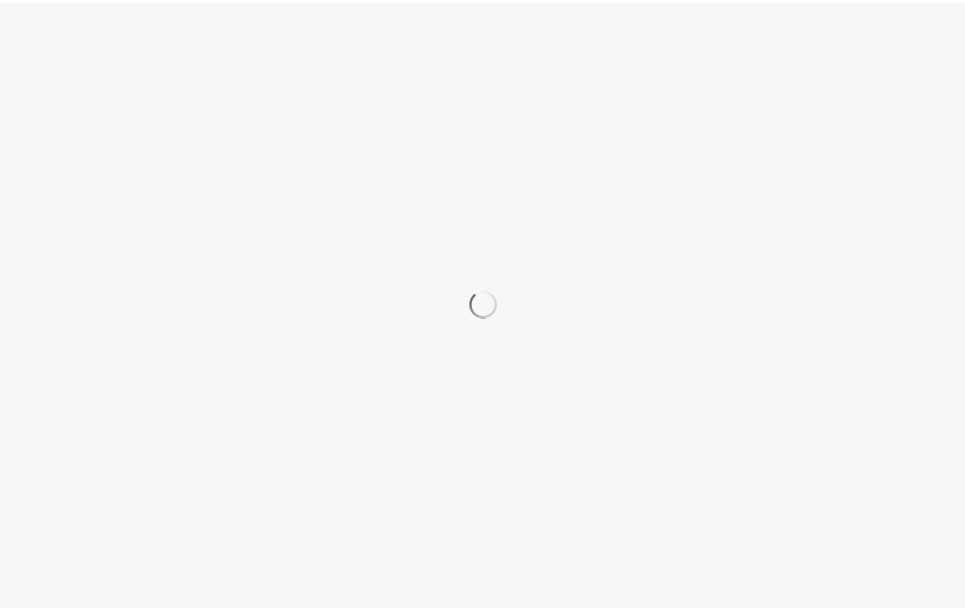
暂时不需要直接推理的内容，可以先搬到上下文窗口之外，例如文件系统、状态对象、日志文件、外部数据库等；上下文里只保留一条短短的摘要、路径或索引。等模型真的需要细节时，再通过这个索引把原文找回来。



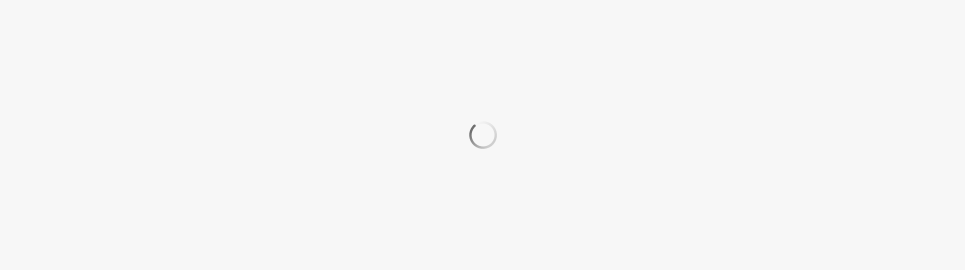
可以用一个很日常的例子理解：你不会把电脑里所有软件说明书都背下来，也不会把所有命令的完整文档贴在桌面上。普通用户会打开应用程序列表，程序员会用 ls /usr/bin 看有哪些命令；真正要用某个命令时，再输入 xxx --help 查看具体参数。Agent 也是类似的：它不一定要在上下文里塞满所有工具说明，只要知道“有哪些工具、在哪里、怎么查看用法”就够了。

这也是为什么 Claude 的 Agent Skills 可以被理解为一种广义的上下文卸载。Claude Skills 是由指令、脚本和资源组成的文件夹，Claude 会根据任务动态加载相关技能；官方说明里也提到，这种 progressive disclosure 可以避免上下文窗口过载。也就是说，技能不是一开始全部塞进上下文，而是“需要时再展开”。(**What are Skills? | Claude Help Center**)

Manus 的思路也很典型。Peak 在 Manus 的 context engineering 文章中明确提出“把文件系统当作上下文”：网页、PDF、文档这类 observation 可能非常大，长输入不仅昂贵，还可能让模型性能下降；因此，Manus 更倾向于把可恢复的信息放到文件系统里，例如保留 URL 或文件路径，而不是让完整内容长期占据上下文窗口。(**Manus**)



Claude Code 的压缩实践也类似：保留架构决策、未解决 bug 和实现细节，清理冗余工具输出，让 Agent 能在更短的上下文里继续工作。(**Anthropic**)



用一句话概括：

上下文卸载就是把 Agent 的工作记忆变轻：眼前只放当前要用的信息，细节放在外部，靠索引随时找回。

3.2 为什么单独用上下文卸载或无限画布都不够？

上下文卸载解决了“信息太长”的问题，却不一定能解决“结构丢失”的问题。大量摘要和文件路径如果只是线性排列，Agent 仍然很难判断任务走到哪一步、哪些信息属于同一个子任务、哪些结果之间存在依赖关系。

如果我们只是把工具返回、网页内容、文件读取结果都保存到外部，然后在上下文里留下摘要和路径，Agent 确实不会被大段内容撑爆。但随着任务变长，它看到的可能变成一串这样的记录：

```
搜索了港大学费，原文见 file_001
搜索了港中文学费，原文见 file_002
搜索了港科大学费，原文见 file_003
读取了项目要求，原文见 file_004
生成了第一版对比表，原文见 file_005
```

这些记录每一条都不长，但问题是：它们仍然是线性的。Agent 需要自己判断哪些信息属于同一个子任务，哪些是前置步骤，哪些已经完成，哪些还需要继续。如果任务只有三五步，这还可以接受；但如果任务有几十次工具调用、多个分支和多次回退，单纯的摘要列表就会变得越来越难用。

可以用一个简单图表示：

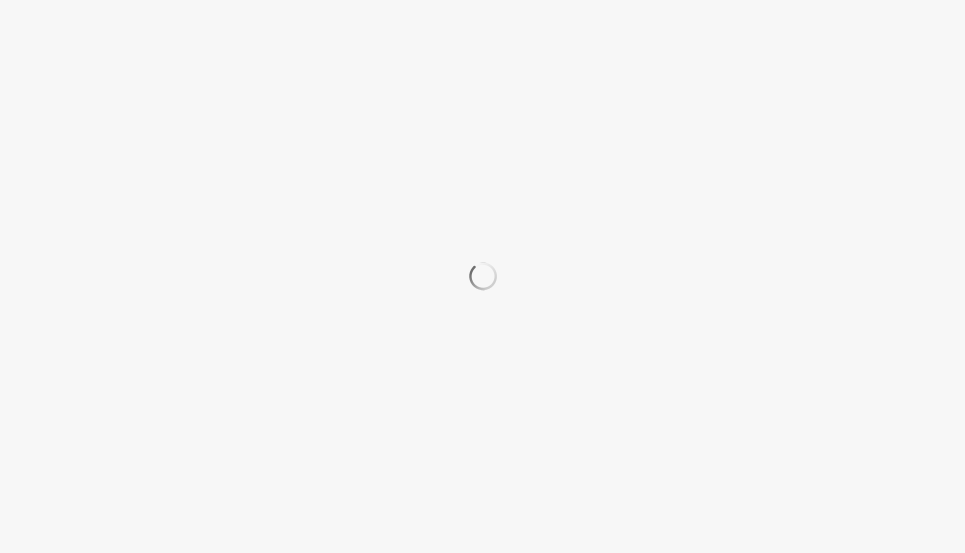
这就像把一堆文件都收进了档案柜，但没有目录、没有标签、没有项目看板。资料没有丢，但你要重新理解它们之间的关系，仍然很费力。

反过来，如果只使用 Mermaid 无限画布，也会有另一个问题：信息可能被压缩得太狠。

Mermaid 画布很适合表达任务结构，比如“先搜索三所学校，再统一学费口径，最后生成表格”。但它本质上是任务骨架，不适合承载所有原始细节。一个节点可以告诉 Agent：

```
N2: 搜索港大学费，status: done
```

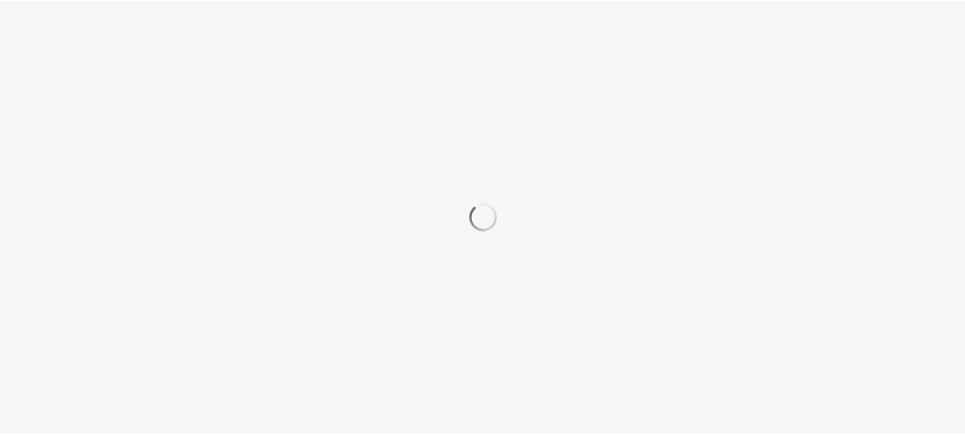
但它不会完整保存搜索结果里的页面内容、引用来源、字段差异、异常情况和原始证据。这样一来，当 Agent 后面需要核对细节时，可能不得不重新搜索、重新读文件、重新调用工具。



所以，单独使用这两种方法都会有缺口。

上下文卸载像“档案柜”，它能保存完整材料，但如果缺少结构，Agent 只看到一堆文件和摘要；Mermaid 无限画布像“任务地图”，它能保存任务结构，但如果没有底层原文支撑，Agent 需要细节时就可能找不到证据。

真正适合长任务的短期记忆压缩，应该同时保留两件事：**细节可恢复，结构不丢失。**



这也是 TencentDB Agent Memory 采用组合方案「**短期记忆压缩 = 上下文卸载 + Mermaid 无限画布**」的原因：前者保存重点细节，后者保存结构。

3.3记忆的折叠：从工具结果到任务画布的具体实践

在实际运行中，每一次工具调用结束后，系统不会把完整结果长期留在上下文里，而是把同一份信息拆成三种形态，分别放到不同位置。

完整结果会被写入 refs/*.md，作为原始证据保存；工具调用会被压缩成一条 JSONL 记录，写入 offload-.jsonl；而当前任务的推进状态，则会被更新到对应的 Mermaid 画布中。

简单来说，一次工具调用结束后，系统会形成这样一条链路：

```
完整 tool result
  ↓
```

```
refs/*.md                      保存原文
  ↓
offload-<sessionId>.jsonl      保存工具调用级 summary
  ↓
mmds/<task>.mmd                保存任务节点级 summary
  ↓
当前上下文                     注入 active MMD 或 history metadata
```

这里最容易误解的是 MMD。它不是一个只有节点标题的空图，也不是单纯的流程骨架。在真实使用中，一个 MMD 节点更像一个“任务卡片”，里面会包含节点名称、状态、摘要和时间戳。

例如，一个真实节点可能长这样：

```
003-N4["timeseries-module-structure<br/>status: done<br/>summary: 列出 timeseries 目录，发现
```

渲染出来后，它不是一句简单的“看过 timeseries 模块”，而是一张可读的任务卡片：Agent 能直接看到这个节点做完了什么、结论是什么、是什么时候完成的。

也就是说，MMD 节点保存的是**任务步骤级摘要**。它回答的是：“这个子任务推进到了哪里？得到什么阶段性结论？”

而 JSONL 保存的是**工具调用级摘要**。它回答的是：“这一次具体工具调用做了什么？完整结果在哪里？”

例如，一条 JSONL 记录可以是：

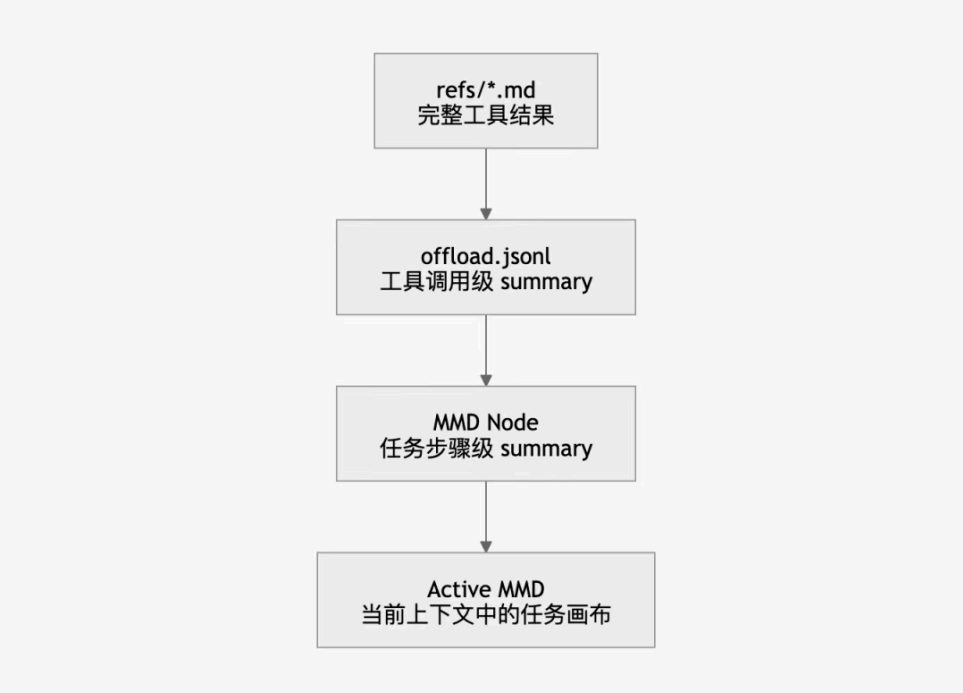
```
{
  "timestamp": "2026-04-12T21:00:33.872+08:00",
"node_id": "002-N3",
"tool_call": "exec({\"command\":\"python3 ~/.openclaw/workspace/skills/sogou-search/sogou
"summary": "获取到北大、清华、人大、复旦等34所自划线院校的2025年复试分数线公布信息，但搜索结果摘要中未
"result_ref": "refs/2026-04-12T21-00-33-872p08-00.md",
"tool_call_id": "toolu_bdrk_01HHx9qNXgXmWUcuT1grEnDC",
"score": 5,
"offloaded": true
}
```

这条记录里，summary 让 Agent 快速理解这次调用的结果，result_ref 指向完整原文，node_id 把这条工具调用挂到某个 MMD 节点上。一个 MMD 节点可以对应多条 JSONL 记录，因为一个任务步骤通常不是一次工具调用就能完成的。

例如，002-N3 这个节点可能叫“搜索复试分数线”。它在 MMD 中只保留一个聚合后的节点摘要，但在 JSONL 中，背后可能挂着多次搜索、网页读取和结果核对。

```
MMD 节点: 002-N3 搜索复试分数线
  ├─ JSONL 记录 1: 搜索自划线院校复试分数线
  ├─ JSONL 记录 2: 打开北大公告页面
  ├─ JSONL 记录 3: 读取清华招生 PDF
  └─ JSONL 记录 4: 核对人大新闻传播专业代码
```

对应的实际结构可以理解为：



这样做的好处是，Agent 平时只需要看 MMD 节点，就能知道任务进度和阶段性结论；如果节点摘要不够，再通过 node_id 找到更细的 JSONL 记录；如果 JSONL 摘要仍然不够，再通过 result_ref 读取完整原文。

总的来说，折叠的存储架构全**部依赖文件系统：**

原文存在 refs/*.md，工具摘要存在 JSONL，任务状态存在 MMD。

3.4记忆的展开：从任务画布回到原始证据的具体实践

在这套方案里，压缩不是一次性把所有历史变成一段 summary，而是逐级折叠。信息会根据上下文压力和任务状态，从完整原文逐步降级为 JSONL、MMD，再降级为 metadata。

可以概括为四层：

```
Level 0: Raw 原文
完整 tool result, 存 refs/*.md
Level 1: JSONL Summary工具调用级摘要, 存 offload-<sessionId>.jsonl``Level 2: MMD Node任务步骤
```

这四层不是互相替代，而是逐层变轻。refs 最完整，但最重；JSONL 更短，适合快速检索工具调用；MMD 更抽象，适合让 Agent 理解任务进度；metadata 最轻，适合在上下文里保留

历史任务入口。

当上下文压力不高时，当前任务的 Active MMD 可以留在上下文里。Agent 看到的是一张带摘要的任务画布：

```
003-N2["repo-scan<br/>status: done<br/>summary: 扫描项目目录，确认主要模块分布和测试目录位置<br/
003-N4["timeseries-module-structure<br/>status: done<br/>summary: 列出 timeseries 目录，发现
003-N5["bug-location<br/>status: doing<br/>summary: 初步定位问题可能与 core.py 中 _calculate
```

这种情况下，Agent 不需要翻完整历史，也能知道：项目已经扫过，timeseries 模块结构已经看过，当前正在定位 bug。

但如果上下文压力继续升高，历史任务的完整 MMD 也不一定继续常驻上下文。这时系统会进一步折叠，只保留任务级 metadata，例如：

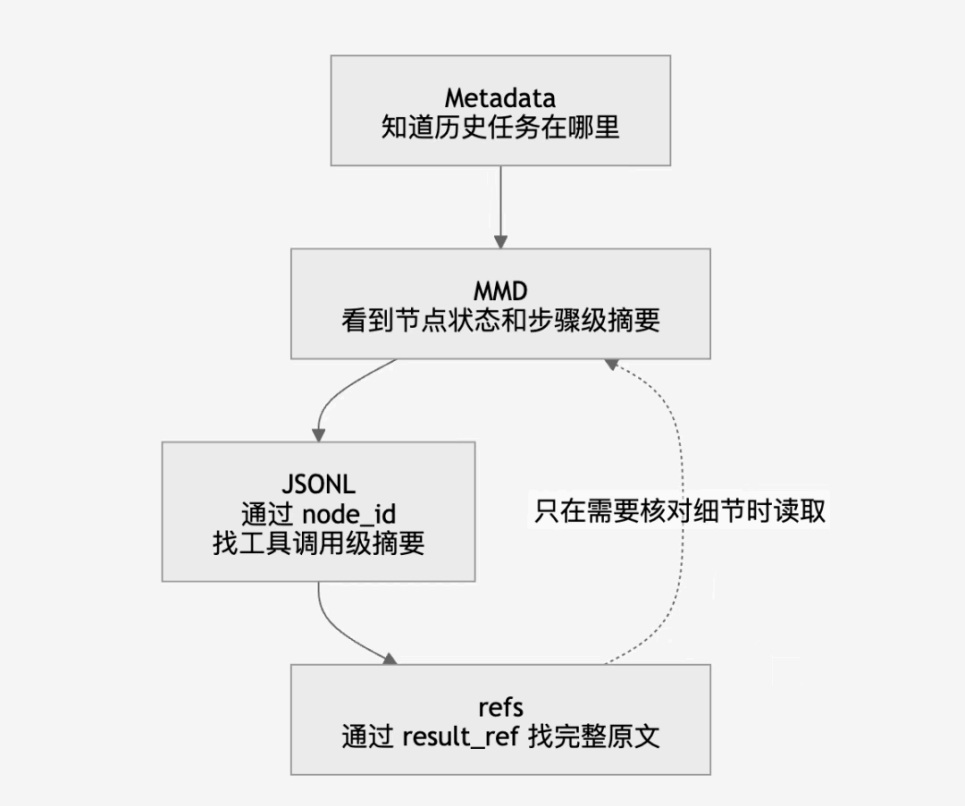
```
{
  "taskGoal": "修复 timeseries 模块中 CompoundModel 的 separability matrix 计算问题",
  "status": "archived",
  "nodeCount": 8,
  "updatedTime": "2026-04-16T22:31:45.120+08:00",
  "mmdFilePath": "mmds/003-timeseries-debug.mmd"
}
```

这条 metadata 不保留细节，但它保留了一个很重要的入口：mmdFilePath。只要这个路径还在，Agent 就能重新打开完整 MMD，恢复这个任务的任务地图。

因此，找回过程也是分级的，不需要一次性恢复全部历史。

如果用户说：“刚才那个 timeseries 的 bug 继续看一下。”Agent 可以先从 metadata 里找到对应任务，打开 mmds/003-timeseries-debug.mmd。打开后，它能看到每个节点的状态和 summary，例如 003-N4 已经确认了 timeseries 目录结构，003-N5 正在定位 _calculate_separability_matrix。如果只看 MMD 就够，Agent 可以直接继续执行；如果需要确认某个节点背后的具体工具结果，就用 **node_id和grep工具**去 offload-*.jsonl 里查找对应记录；如果还需要完整文件内容或日志，再读取 JSONL 里的 result_ref。

找回路径可以表示为



这个设计的关键是：MMD 节点本身已经有摘要，所以 Agent 很多时候不需要继续下钻。只有当节点 summary 不够支持下一步判断时，才查 JSONL；只有当 JSONL summary 也不够时，才读 refs 原文。

这样可以避免两种极端：一种是所有信息都塞在上下文里，导致上下文腐烂；另一种是压缩得太狠，导致 Agent 频繁重新调用工具。我们的做法是在中间建立几个可恢复层级，让信息可以根据需要展开。

任务记忆的召回分四层：metadata链接任务入口，MMD链接任务状态，JSONL链接工具摘要，refs链接原始证据。

四、实验设计与结果

4.1 实验设计：把多个任务串成超长 Session

这组实验不是常见的“单题单上下文”评测，而是刻意构造了更接近真实 Agent 使用场景的超长 Session 压力测试。

以 SWEbench 为例，我们没有让 Agent 每次只完成一道题、做完后清空上下文，而是把 50 个 **task 串在同一个 session 中连续执行。**也就是说，Agent 做完第 1 个任务后，会带着前面任务产生的MSG、Tool Use和过程文件等，继续执行第 2 个、第 3 个……直到完成这一组连续任务。SWEbench 500 题可以理解为由多组这样的长 Session 组成。

其他几个测试集也采用了类似设计：Toolathlon、WideSearch 和 AA-LCR 都不是单独测试一个 isolated task，而是把多个任务放在连续会话中执行，让上下文不断累积，从而观察 Agent 在长时间运行后是否还能保持任务目标、调用正确工具，并避免被旧信息干扰。

这个实验设计会让绝对成绩比市面上一些“单题独立上下文”的公开结果更低，因此不适合直接和外部榜单做横向对比。但它更适合验证本文真正关心的问题：

当 Agent 连续执行大量任务、上下文不断膨胀时，短期记忆压缩能否让它更省 Token、更少遗忘、更稳定地完成任务？

因此，**下面的实验重点不在于绝对分数（可能会比市面上相同测试集测试分数更低），而是在相同的超长 Session 设置下，对比无插件和有短期记忆压缩插件之间的相对变化。**下文中的“Token 节省”统一用正数表示，即相比无插件基线减少了多少 Token。

4.2 总体结果：Token 降低的同时，任务完成率没有下降

从整体结果看，短期记忆压缩方案在四类评测中都体现出比较稳定的收益：一方面，Token 消耗明显下降；另一方面，任务完成率或准确率没有下降，反而在多数场景中有所提升。

评测集	任务类型	加插件后成功率相对变化	Token 节省
SWEbench	代码修复任务	完成率相对提升 9.93%	最高节省 33.09%
Toolathlon	复杂长任务	通过率从 20% 提升到 35%	最高节省 26.18%
WideSearch	网页搜索任务	通过率相对提升 51.52%	最高节省 61.38%
AA-LCR	长文总结与分析	准确率相对提升7.95%	总 Token 节省 30.98%

这个结果说明，短期记忆压缩并不是简单地“删上下文”。如果只是粗暴删除信息，Agent 很容易在长任务后半段丢失目标、重复搜索、重复读取文件，甚至调用错误工具。而我们的方案把不同粒度的信息放到了不同位置：原文被卸载到外部文件，工具调用被压缩成 JSONL 摘要，任务状态被写入 Mermaid 画布，历史任务入口则保留在 metadata 中。

因此，当前上下文变轻了，但任务链路没有断。

4.3 SWEbench：代码任务中同时提升完成率与 Token 效率

在 SWEbench 500 题测试中，主 Agent 使用 4.5-haiku。无插件时完成率为 58.4%；加入短期记忆压缩插件后，不同 Offload 模型下完成率提升到 61.8%–64.2%。

方案	完成率	相对无插件提升	Token 使用	Token 节省
无插件	0.584	•	3474.1	•
插件 - Opus 4.6	0.632	8.22%	2399.4	30.93%
插件 - MiniMax 2.7	0.618	5.82%	2324.4	33.09%
插件 - GLM 5.1	0.642	9.93%	2375.4	31.63%

代码任务会产生大量文件读取、测试日志、错误输出和中间修改记录。如果这些内容长期堆在上下文中，模型很容易被旧日志和无关代码片段干扰。短期记忆压缩插件将完整结果移出上下文，只保留摘要、节点状态和索引路径，使 Agent 更容易维持对 bug 定位过程的理解。

从结果看，插件不仅节省了约 31%–33% 的 Token，也带来了 5.82%–9.93% 的完成率相对提升。这说明在连续代码任务中，压缩并没有削弱 Agent 的能力，反而通过减少上下文噪

音，提高了任务稳定性。

4.4 Toolathlon：复杂长任务中，通过率从 20% 提升到 35%

Toolathlon 更接近真实复杂任务场景。它的任务数量较少，但每个任务链路更长，对上下文连续性、工具调用稳定性和任务状态管理要求更高。本组实验中，主模型使用 Opus 4.6。

方案	通过	未通过	通过率	通过率提升	总 Token	Token 节省
无 Offload	4	16	20%	•	50.81M	•
Offload: Opus 4.6	6	14	30%	50.00%	41.17M	18.97%
Offload: MiniMax 2.7	7	13	35%	75.00%	37.51M	26.18%
Offload: GLM 5.1	7	13	35%	75.00%	44.23M	12.95%
Offload: 自部署 GLM 5.1	7	13	35%	75.00%	48.28M	4.98%

在这个评测中，加入 Offload 后，通过数从 4 个提升到 6–7 个，通过率从 20% 提升到 30%–35%。其中 MiniMax 2.7 作为 Offload 模型时，Token 节省最高，达到 26.18%。

这组结果说明，在复杂长任务里，失败并不总是因为模型完全不会做，而往往是因为任务做到后半段时，上下文已经变得很重，模型开始忘记前面的判断、重复执行工具，或者无法准确接上之前的任务状态。短期记忆压缩的作用，是把已经完成的中间过程折叠起来，让 Agent 保留任务结构，而不是背负全部原始信息继续前进。

需要说明的是，Toolathlon 只有 20 个任务，样本量相对较小，因此更适合作为方向性验证。它说明这套方案在复杂长任务上有明显潜力，但后续仍需要更大规模任务集进一步验证。

4.5 WideSearch：网页搜索任务中 Token 节省最明显

WideSearch 评测包含 200 个网页搜索任务，主模型为 4.5-haiku。网页搜索任务天然会产生大量网页正文、搜索结果和重复页面内容，因此非常适合观察上下文卸载的效果。

方案	通过率提升	Token 消耗	Token 节省
无插件	•	221.51M	•
插件 - Opus	51.52%	85.64M	61.38%
插件 - GLM 5.1	30.88%	89.87M	59.39%

从结果看，加入插件后，Token 消耗显著下降。Opus Offload 方案将 Token 从 174.31M 降至 63.46M，节省 63.59%；GLM 5.1 Offload 方案节省 48.44%。

通过率方面，Opus Offload 从 8.5% 提升到 12%，相对提升 41.18%；GLM 5.1 Offload 提升较小，从 8.5% 到 9%。这说明在网页搜索类任务中，压缩和卸载对 Token 成本的改善非常明显，但最终任务效果仍会受到 Offload 模型质量、摘要质量和搜索路径选择的影响。

整体来看，这组结果验证了一个重要判断：对于长网页、多轮搜索和多来源整合场景，完整内容不应该长期滞留在上下文中。更合理的做法是把网页原文存到外部文件，把阶段性发现挂到任务画布上，让 Agent 按需回看。

4.6 AA-LCR：长文总结与分析中，总 Token 节省 31%

AA-LCR 是 800 题文章总结与分析任务，主模型为 4.5-haiku。其中有插件和无插件各测试 400 题。

方案	准确率	主模型 Token	总 Token
有插件	190/400（47.5%）	70,944,374	77,283,611
无插件	176/400（44.0%）	112,029,517	112,029,517

加入插件后，准确率从 44.0% 提升到 47.5%，多答对 14 题。主模型 Token 从 112,029,517 降至 70,944,374，节省约 36.67%。如果把 Offload 模型本身消耗的 Token 也计算在内，总 Token 为 77,283,611，相比无插件仍节省约 31%。

这说明在长文总结和分析任务中，短期记忆压缩并没有破坏模型对文章内容的理解，反而可能通过减少上下文冗余，让主模型更稳定地聚焦在文章结构、问题目标和关键证据上。

4.7 消融实验：MMD的作用验证

为了验证 Mermaid 无限画布的作用，我们进一步做了消融实验：只保留上下文卸载能力，不使用 MMD 任务画布。

在 SWEbench 上，仅有上下文卸载时，Token 节省约 15%，成绩提升约 5%。而完整方案中，Token 节省可以达到约 31%–33%，成绩提升约 5.82%–9.93%。

方案	SWEbench 成绩变化	Token 节省	说明
无插件	基线	基线	完整工具结果留在上下文中
仅上下文卸载	提升约 5%	节省约 15%	原文被移出上下文，但任务结构保留较弱
上下文卸载 + MMD	提升约 9.9%	节省 31%–33%	原文可恢复，任务结构也被保留

这个结果说明，上下文卸载本身确实有效，但它主要解决的是“内容太长”的问题。它能把完整工具结果从上下文中移走，降低 Token 压力；但如果没有 MMD，Agent 看到的仍然可能是一串零散摘要和文件路径，不一定能快速理解这些信息之间的任务关系。

Mermaid 无限画布进一步解决的是“结构容易丢”的问题。它把工具调用摘要组织成带状态、摘要和时间戳的任务节点，使 Agent 能看到任务进行到哪里、哪些步骤已经完成、哪些节点还需要继续处理。

因此，完整方案的收益来自两部分：

上下文卸载：减少长工具结果对上下文窗口的占用 Mermaid 无限画布：减少任务结构在压缩过程中的损失 这也解释了为什么完整方案相比“仅上下文卸载”能进一步提升 Token 效率和任务完成率。

4.8实验分析小结

从四组实验可以看到，TencentDB Agent Memory 的短期记忆压缩方案在超长 Session 场景中表现出两个稳定特征。

第一，Token 消耗显著下降。WideSearch 节省 61.38%，SWE-bench 节省 33.09%，AA-LCR 节省 30.98%。

第二，任务完成率没有因为压缩下降，反而在多个评测中提升。WideSearch 通过率从 33% 提升到 50%（相对 +51.52%），SWE-bench 完成率从 58.4% 提升到 64.2%（相对 +9.93%），AA-LCR 准确率从 44.0% 提升到 47.5%（绝对 +3.5pp）。

更重要的是，这些结果是在多任务连续 Session 中得到的，而不是单题清空上下文的独立评测。因此，它们更能反映生产级 Agent 在长时间运行中的真实压力：上下文会不断膨胀，工具结果会不断累积，旧任务和新任务会相互干扰。

短期记忆压缩的价值，正是在这样的场景中体现出来：

压缩不是让 Agent 少知道，而是让 Agent 少背负；信息可以离开上下文窗口，但不能离开 Agent 的可达范围。

五、产品落地与开源

TencentDB Agent Memory 目前的产品功能布局分为两层：**短期任务记忆和长期个性化记忆。**

这样分，是因为 Agent 面临**两类不同的“遗忘”问题**：一类发生在任务中，做着做着上下文变长、步骤变乱、目标丢失；另一类发生在任务之间，下次再见面时，Agent 不知道用户过去的偏好、习惯和长期目标。

短期记忆解决的是：怎么集中注意力完成当前任务。

长期记忆解决的是：下一次再见面时，Agent 怎么更懂你。

5.1 短期记忆压缩：适合长任务和高工具调用场景

回到最开始的问题：为什么要讨论符号化压缩、Mermaid 无限画布和上下文卸载？

价值可以概括为三点：

第一，更省 Token。长网页、长日志、长工具输出不再长期占据上下文窗口。

第二，更稳完成长任务。Mermaid 画布保留任务结构，让 Agent 知道任务做到哪里、下一步该做什么。

第三，更容易复盘和恢复。原文、摘要、节点和 metadata 之间有清晰索引，任务断点和失败原因都可以追溯。

短期记忆压缩能力目前已经在正式产品中上线，但从**超长 Session 实验结果**来看，它尤其适合**办公提效、创作、研究和编程这类长任务和多任务场景。**

这类任务通常有三个特点：**资料多、步骤长、反复改**。Agent 可能需要连续搜索网页、读取文件、运行工具、生成中间结果，再根据用户反馈不断调整。如果所有网页内容、代码日志、文件片段、中间版本和工具输出都长期堆在上下文里，任务越做越重，模型也更容易忘记原始目标、混淆版本，或者重复执行已经做过的步骤。

短期记忆压缩的目标，就是让 Agent 在这类任务中保持轻量和连续：原始资料可以先卸载到外部，关键过程被压缩成摘要，任务结构则保存在 Mermaid 画布中。这样，Agent 不需要一直背着所有历史信息前进，但在需要时仍然能逐层找回。

比如在**办公与行业研究场景（如WideSearch测评集）**中，Agent 往往需要连续阅读网页、公司资料、新闻报道和研报片段，并最终输出结构化分析。短期压缩可以帮助它保留调研路径和阶段性结论，减少重复搜索和材料混乱。

在**报告写作与内容创作场景（如AA-LCR测评集）**中，Agent 可能先收集资料，再生成大纲，然后多轮改写。短期压缩可以记录不同版本的结构调整、用户反馈和关键引用来源，让后续修改不容易丢失上下文。

在**编程与代码修复场景（如SWEbench测评集）**中，Agent 会不断读取文件、运行测试、查看报错、修改代码。短期压缩可以保留 bug 定位路径、关键文件摘要和测试结果引用，避免后半程被旧日志和无关片段干扰。

在**复杂多步骤任务（如Toolathlon测评集）**中，用户可能中途切换目标，或者让 Agent 同时处理搜索、分析、生成和修改。短期压缩可以把不同子任务拆成节点，让 Agent 知道哪些已经完成、哪些还在进行、哪些需要恢复。

除了以上推断，我们也非常期待能在正式上线后收到更多真实用户的反馈。

5.2 长期个性化记忆：让 Agent 复用用户偏好

除了短期记忆压缩，TencentDB Agent Memory 也会提供 长期个性化记忆模块。它同样延续“渐进式披露”的思想：不把所有用户历史一次性塞进上下文，而是将用户偏好、长期目标、任务习惯和历史经验分层沉淀，在合适的时候按需注入。



在包含 6000+ 条消息、589 道题的 PersonaMem 评测集上，**接入 TencentDB Agent Memory 后，回答准确率相比原生“龙虾”记忆相对提升 59%，从不到 48% 提升到 76%。**说明在需要理解长期用户画像、偏好和历史行为的任务中，长期记忆模块可以明显提升 Agent效果。

在 PersonaMem 这类专门测试个性化记忆的评测集中，**长期记忆也可以带来非常高的 Token 节省，部分场景下可接近 90%。**但我们认为这类评测和**真实使用方式存在差异**：用户在日常使用中并不会一直连续追问个人画像问题，而是会在办公、研究、编程和创作任务中穿插使用记忆。因此，我们没有把这个测试集的token节省结果统计到插件效果中。

除了评测数据，**此能力已经上线和开源，我们已经能在一些真实场景中看到用户反馈。**

例如，做量化交易的用户在工作中经常会有稳定偏好：希望回测时默认考虑交易成本，希望比较 LSTM、Transformer、Chronos 等模型，希望输出里包含阈值设置、收益曲线、成本前后对比和风险提示。长期记忆可以把这些偏好沉淀下来，后续用户再次做类似任务时，Agent 不需要每次都重新询问一遍，而是可以自动复用之前的分析口径和输出习惯。

再比如研究类任务中，用户可能长期关注某个方向，例如 Agent Memory、向量数据库评测、代码搜索工具或上下文工程。长期记忆可以记录用户已经研究过哪些问题、倾向什么判断框架、偏好什么表达风格，让下一次讨论更快进入上下文。

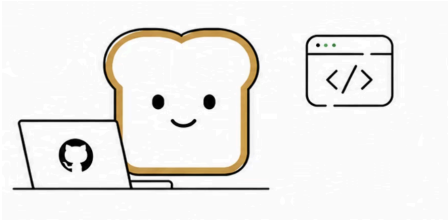
5.3产品化与开源计划

- 目前，产品可作为插件在OpenClaw框架中使用，为“龙虾”装上不断进化的“第二大脑”。也在适配Hermes等其他Agent框架。

- 长期记忆功能已经上线Qclaw、Lighthouse和ClawPro等产品，支持一键启用，短期记忆功能已上线。

- 整套方案即将在 GitHub 开源，我们诚挚邀请开发者社区一同探讨记忆系统设计中的工程取舍，也期待你通过 Issue 提出反馈与建议，共建下一代 Agent 记忆基础设施，项目Github链接： **https://github.com/Tencent/TencentDB-Agent-Memory**

- 如果 TencentDB Agent Memory 对你有所帮助，欢迎为项目点亮 ⭐ 支持。





腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 9428

留言

写留言

还没有留言，来写首评